

File Path Traversal

Traversal Sequences Stripped Non-Recursively — Nivel Apprentice

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Materia	CNO IV – Seguridad Informática
Laboratorio	LAB 05 · File Path Traversal – Non-Recursive Stripping Bypass
Nivel	Apprentice
Plataforma	PortSwigger Web Security Academy
Docente	Mtro. Servando López Contreras
Fecha	22 de septiembre de 2026

Índice

1. Marco Teórico y Fundamentación Técnica
2. Identificación del Laboratorio
3. Objetivo del Laboratorio
4. Explicación Técnica de la Vulnerabilidad
5. Procedimiento Paso a Paso
6. Capturas de Pantalla Obligatorias
7. Explicación Técnica del Payload
8. Mitigación Recomendada
9. Conclusión

1. Marco Teórico y Fundamentación Técnica

El Path Traversal es una vulnerabilidad de seguridad web que permite a un atacante acceder a archivos arbitrarios del servidor mediante la manipulación de parámetros que construyen rutas del sistema de archivos. Cuando las aplicaciones implementan filtros para mitigar este tipo de ataque, es fundamental que dichos filtros sean completos y aplicados de forma recursiva. La variante documentada en este laboratorio explota un filtro que elimina secuencias de traversal (../) de forma no recursiva, lo que deja expuesto el sistema a un patrón de bypass específico.

El stripping no recursivo (eliminación en una sola pasada) es una técnica de mitigación incompleta: el filtro detecta y elimina la secuencia ../ una única vez del valor del parámetro, pero no vuelve a analizar el resultado para verificar si el proceso de eliminación generó nuevas secuencias de traversal. Un atacante puede construir un payload que, tras la eliminación, produce exactamente la secuencia de traversal que el filtro intentaba bloquear.

Estándar	Identificador	Descripción
CWE	CWE-22	Improper Limitation of a Pathname to a Restricted Directory
OWASP Top Ten	A01:2021	Broken Access Control — validación de entradas insuficiente
CVSS v3.x	Base Score: 7.5 (High)	Acceso no autorizado a archivos del sistema sin autenticación
CVSS Vector	AV:N/AC:L/PR:N/UI:N	Explotable remotamente, sin privilegios, sin interacción del usuario

La raíz de la vulnerabilidad reside en la implementación incorrecta de la medida de seguridad: filtrar por lista negra sin aplicar el filtro de forma exhaustiva y recursiva equivale a no filtrar. El único enfoque robusto es la canonicalización de la ruta resultante combinada con la verificación de confinamiento al directorio base, independientemente de la técnica de manipulación empleada por el atacante.

2. Identificación del Laboratorio

Campo	Detalle
Nombre del laboratorio	File Path Traversal — Traversal Sequences Stripped Non-Recursively
Nivel	Apprentice
Tipo de explotación	Path Traversal / Non-Recursive Stripping Bypass
Plataforma	PortSwigger Web Security Academy
Posición en la ruta	LAB 05 — Road to Hall of Fame
Herramienta principal	Burp Suite Community Edition v2026.8
Protocolo explotado	HTTP/2

Campo	Detalle
Vector de ataque	Parámetro filename del endpoint /image
Técnica de bypass	Payload anidado que reconstruye ../ tras el stripping no recursivo

3. Objetivo del Laboratorio

Objetivo general:

Demostrar de forma práctica y documentada la explotación de una variante de Path Traversal en la que la aplicación elimina secuencias ../ del parámetro filename de manera no recursiva. Mediante la construcción de un payload anidado (....//....//....//etc/passwd) que reconstruye las secuencias de traversal tras el proceso de filtrado, se recuperará el contenido del archivo /etc/passwd del servidor objetivo.

Objetivos específicos

- Identificar la petición HTTP vulnerable con el parámetro filename (GET /image?filename=37.jpg) en el historial HTTP de Burp Suite.
- Interceptar la petición y enviarla al módulo Repeater para su modificación controlada.
- Construir el payload//....//....//etc/passwd que evade el filtro no recursivo y genera una ruta de traversal válida.
- Verificar que el servidor responde con HTTP/2 200 OK y devuelve el contenido de /etc/passwd.
- Obtener la confirmación 'Lab Solved' de la plataforma PortSwigger.

4. Explicación Técnica de la Vulnerabilidad

La aplicación del laboratorio es una tienda en línea que sirve imágenes de producto a través del siguiente endpoint:

```
GET /image?filename=37.jpg HTTP/2
Host: <lab-id>.web-security-academy.net
```

La aplicación implementa un filtro que detecta y elimina la secuencia ../ del valor del parámetro filename. Sin embargo, este proceso se realiza en una única pasada (no recursivamente): el filtro recorre el valor de izquierda a derecha, elimina cada ocurrencia de ../ que encuentre, y no vuelve a revisar el resultado. Esta característica es la que hace posible el bypass.

Mecanismo del bypass: cómo funciona el payload//

El bloque fundamental del payload es//. Para entender por qué funciona, es necesario analizar qué ocurre cuando el filtro procesa esta cadena de caracteres:

```

Payload original:  ....//
Caracteres:      . . . . / /
Posición:        0 1 2 3 4 5

El filtro busca '../' (punto-punto-barra).
Encuentra '../' en las posiciones 2, 3, 4 → elimina esos 3 caracteres.

Resultado tras el stripping:
  Quedan: posiciones 0, 1 → '..'
  +      posición 5 → '/'
  =      '../' ← secuencia de traversal reconstruida

```

En resumen: el payload// contiene la secuencia ../ embebida de forma que, al ser eliminada por el filtro, los caracteres restantes forman una nueva ../ que el filtro ya no vuelve a analizar. Con tres repeticiones del bloque (....//....//....//), el payload completo reconstruye ../../etc/passwd tras el filtrado, suficiente para alcanzar la raíz del sistema de archivos del servidor.

```

Payload completo:  ....//....//....//etc/passwd
Tras el stripping: ../  ../  ../  etc/passwd
Ruta resuelta:    /etc/passwd

```

Tabla resumen del comportamiento del filtro

Valor del parámetro	Acción del filtro	Resultado
37.jpg	No contiene ../ — pasa sin cambios	Sirve la imagen correctamente
../../etc/passwd	Detecta y elimina ../ tres veces	Bloquea / sirve ruta errónea
....//....//....//etc/passwd	Elimina ../ embebido en// (no recursivo)	Reconstruye ../../etc/passwd — accede al archivo

5. Procedimiento Paso a Paso

Paso 1: Acceso al laboratorio y configuración de Burp Suite

Se accedió a la plataforma PortSwigger Web Security Academy y se abrió el laboratorio 'File Path Traversal — Traversal Sequences Stripped Non-Recursively'. Se copió la URL asignada al entorno de práctica. Se abrió Burp Suite Community Edition con la configuración predeterminada, con el proxy local activo en 127.0.0.1:8080.

Paso 2: Carga de la página en el navegador de Burp Suite

Desde la pestaña Proxy se abrió el navegador integrado de Burp Suite ('Open Browser'). Se pegó la URL del laboratorio y se esperó a que la tienda en línea cargara completamente. Todo el tráfico HTTP quedó disponible en el historial de Burp Suite.

Paso 3: Identificación del vector de ataque

Se activó el modo Intercept ('Intercept is on') y se recargó la página. En el historial HTTP se identificó la petición GET /image?filename=37.jpg, que corresponde a la carga de una imagen de producto. Esta petición contiene el parámetro filename, que es el vector de ataque. Se hizo clic derecho sobre ella y se seleccionó 'Send to Repeater' (Ctrl+R).

Paso 4: Envío de la petición original al Repeater

La petición fue transferida al módulo Repeater de Burp Suite, donde se visualizó la petición original antes de cualquier modificación. El endpoint y el valor del parámetro fueron confirmados:

```
GET /image?filename=37.jpg HTTP/2
Host: <lab-id>.web-security-academy.net
```

Paso 5: Construcción y envío del payload de bypass

En el módulo Repeater se modificó el valor del parámetro filename, sustituyendo el nombre de la imagen por el payload de bypass no recursivo. Se presionó el botón 'Send':

```
# Petición original:
GET /image?filename=37.jpg HTTP/2

# Petición modificada con payload de bypass:
GET /image?filename=../../../../etc/passwd HTTP/2
```

Paso 6: Análisis de la respuesta

El panel de respuesta del Repeater mostró el código HTTP/2 200 OK y el cuerpo de la respuesta contenía el archivo completo /etc/passwd del sistema operativo del servidor, incluyendo la lista de usuarios del sistema (root, daemon, peter, carlos, entre otros).

Paso 7: Confirmación — Lab Solved

Se apagó el interceptador, se actualizó la página en el navegador integrado y la plataforma PortSwigger mostró el banner de confirmación 'Congratulations, you solved the lab!', validando el éxito del bypass.

6. Capturas de Pantalla Obligatorias

Se presentan las cinco capturas de pantalla que documentan visualmente el proceso completo de explotación, desde la interceptación de la petición hasta la confirmación de resolución del laboratorio.

Captura 1 — Petición interceptada y enviada al Repeater

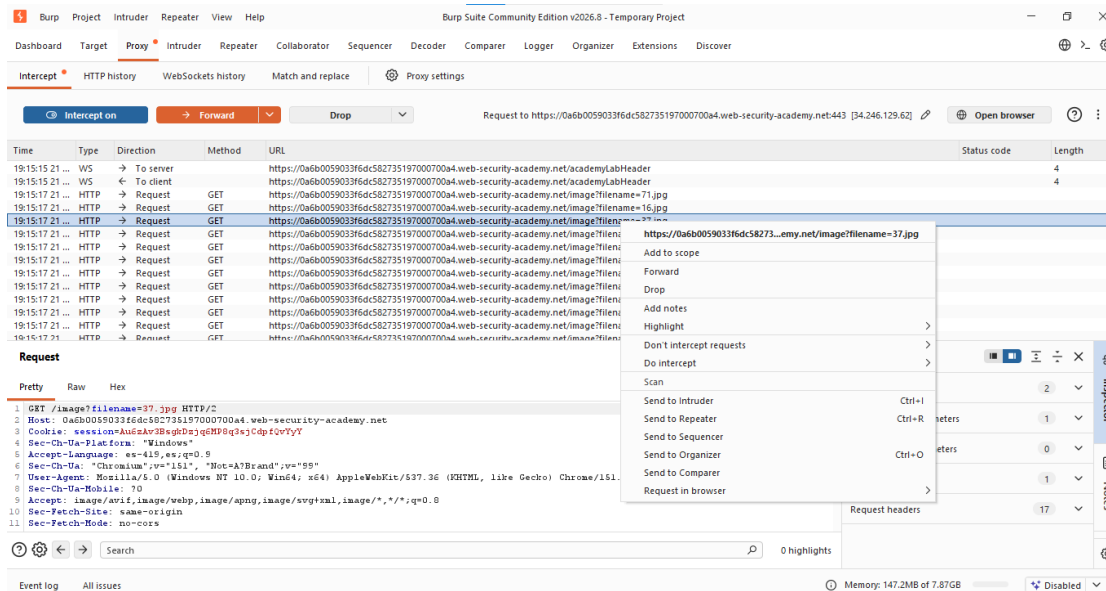


Figura 1. Historial HTTP de Burp Suite con la petición GET /image?filename=37.jpg interceptada (resaltada). Se observa el menú contextual con la opción 'Send to Repeater' seleccionada para transferir la petición al módulo de análisis.

Captura 2 — Petición original en el módulo Repeater

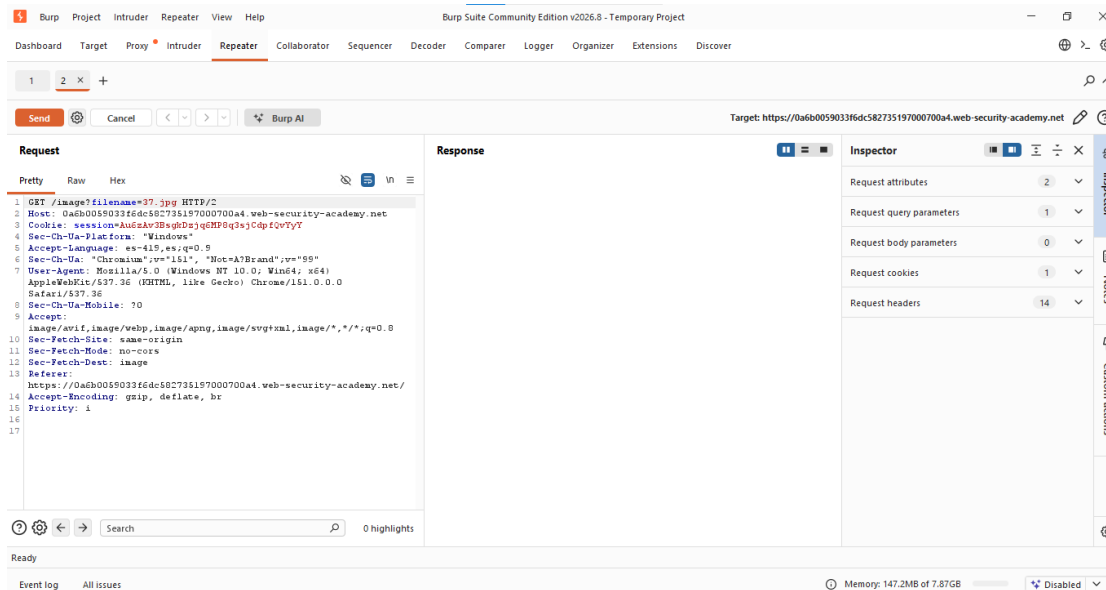


Figura 2. Petición original GET /image?filename=37.jpg HTTP/2 en el módulo Repeater, lista para ser modificada. El panel de respuesta está vacío, confirmando que aún no se ha enviado ninguna petición modificada.

Captura 3 — Payload de bypass enviado al servidor

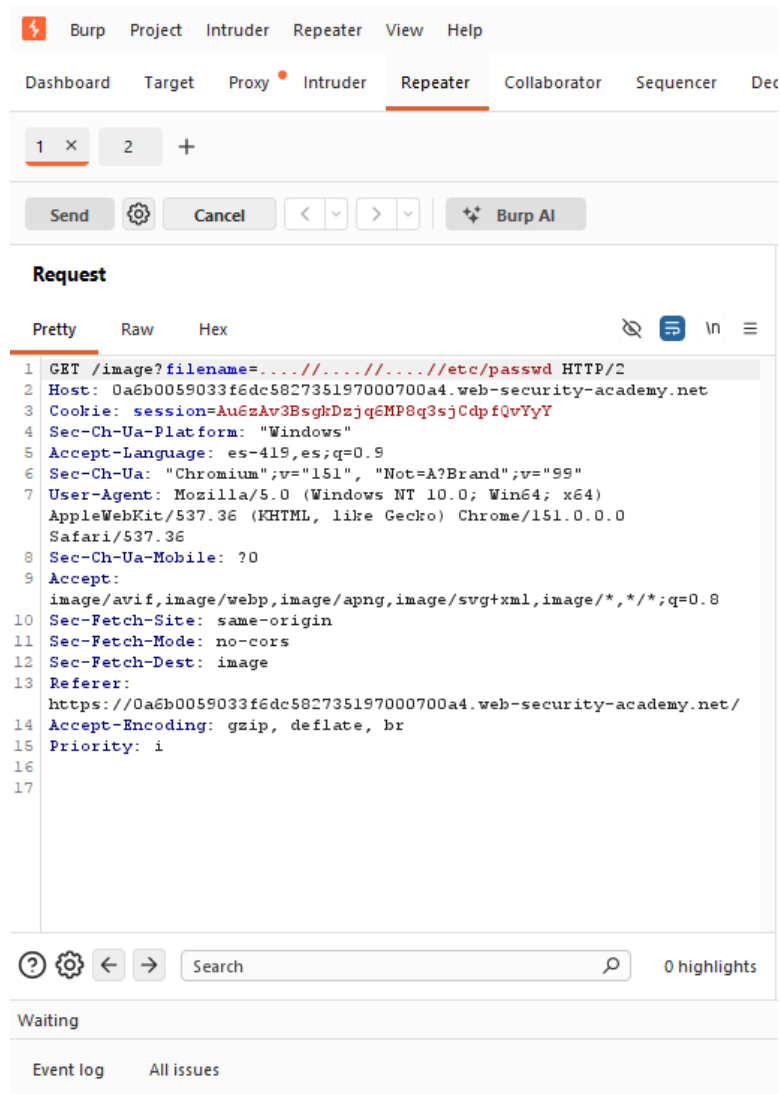


Figura 3. Petición modificada con el payload de bypass: `GET /image?filename=../../../../../../../../etc/passwd HTTP/2`. El estado 'Waiting' indica que la petición fue enviada y el servidor está procesando la respuesta.

Captura 4 — Respuesta del servidor con el contenido de `/etc/passwd`

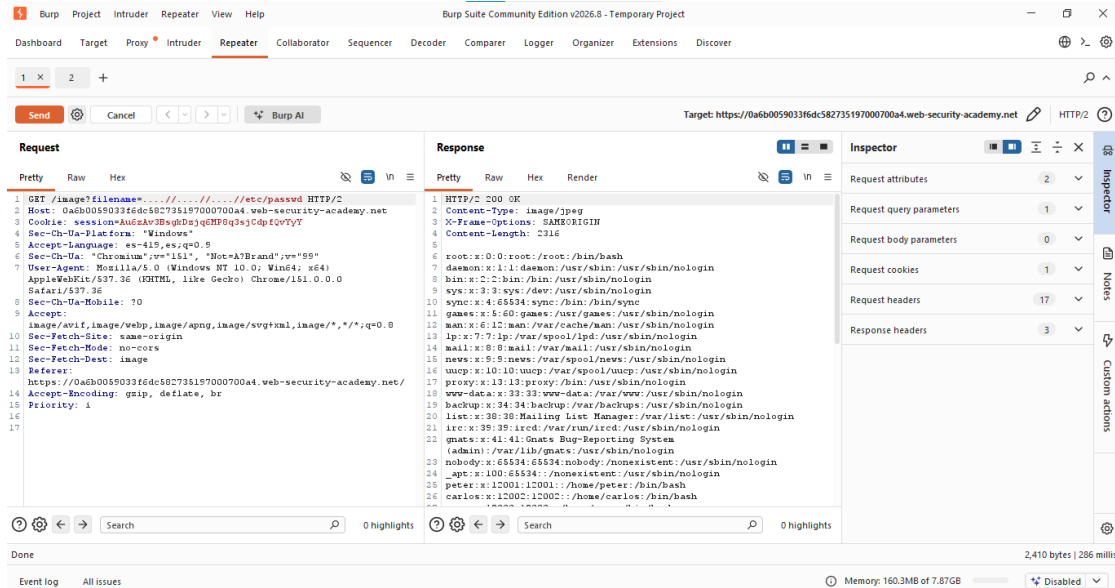


Figura 4. Respuesta HTTP/2 200 OK con el contenido completo del archivo `/etc/passwd`. Se observan los registros de usuarios del sistema: `root`, `daemon`, `bin`, `sys`, `peter`, `carlos` y otros. El payload de bypass eludió exitosamente el filtro no recursivo.

Captura 5 — Confirmación 'Lab Solved' de PortSwigger

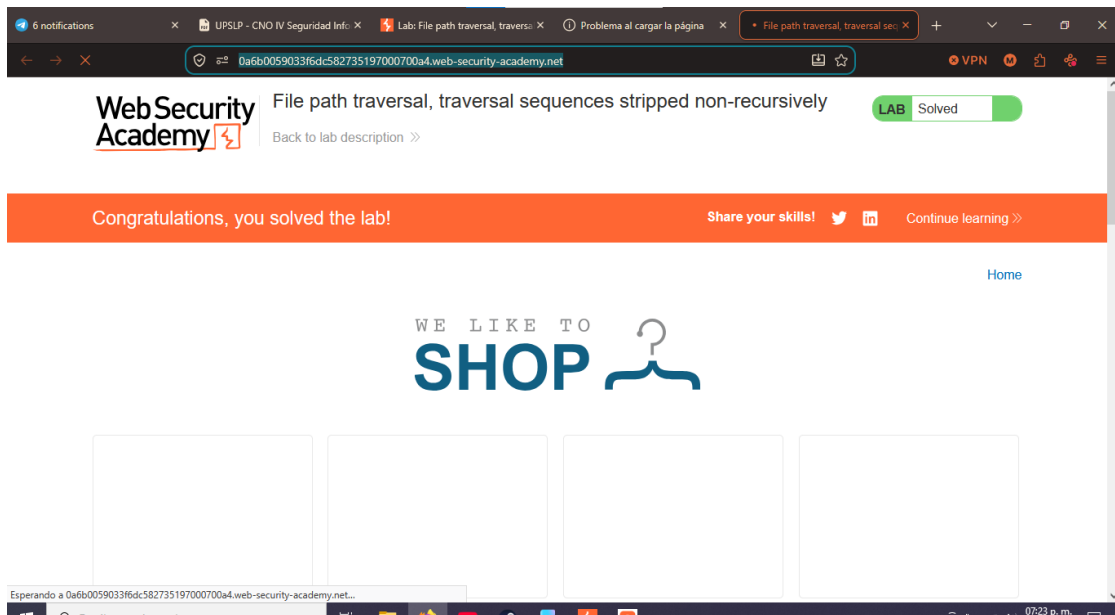


Figura 5. Banner de confirmación: 'Congratulations, you solved the lab!' — File path traversal, traversal sequences stripped non-recursively. Laboratorio resuelto exitosamente.

7. Explicación Técnica del Payload

```
....//....//....//etc/passwd
```

El payload se construye a partir de la repetición de un bloque fundamental (....//), elegido específicamente para que el proceso de stripping no recursivo lo transforme en una secuencia de traversal válida:

Bloque	Tras el stripping no recursivo	Función en la ruta
....//	../	Sube un nivel en la jerarquía de directorios
....//	../	Sube un segundo nivel
....//	../	Sube hasta la raíz del sistema de archivos
etc/passwd	etc/passwd (no se modifica)	Ruta al archivo objetivo desde la raíz

¿Por qué// y no otras variantes?

El bloque// funciona porque contiene la secuencia ../ embebida en posiciones internas (posiciones 2-4 de la cadena de 6 caracteres). Al eliminar esas tres posiciones, los dos puntos iniciales (posiciones 0-1) y la barra final (posición 5) se unen formando una nueva ../ que el filtro no vuelve a analizar. Otras variantes como\ (con barra invertida) o / (con espacios) funcionan en plataformas o frameworks distintos con comportamientos de stripping diferentes.

Es importante señalar que el payload podría extenderse con más bloques (....//....//....//....//etc/passwd) sin afectar el resultado, siempre que al menos tres bloques produzcan ../ tras el stripping. Los niveles adicionales de traversal se resuelven de forma inocua una vez que se llega a la raíz del sistema de archivos.

8. Mitigación Recomendada

La vulnerabilidad explotada en este laboratorio tiene su origen en un filtro de seguridad incompleto. Las siguientes medidas, aplicadas de forma combinada, eliminan el riesgo de forma robusta:

8.1 Canonicalización y verificación de la ruta resultante (prioridad crítica)

Esta es la única medida que detiene todos los tipos de bypass de Path Traversal, independientemente de la técnica de ofuscación utilizada. Consiste en resolver la ruta canónica completa antes de acceder al archivo y verificar que permanece dentro del directorio base autorizado:

```
import os

BASE_DIR = '/var/www/images'

def get_image(filename: str):
    # Resolver la ruta canónica (elimina ../, rutas absolutas,
    # doble codificación, caracteres raros, etc.)
    full_path = os.path.join(BASE_DIR, filename)
    canonical = os.path.realpath(full_path)
```

```
# Verificar que la ruta resuelta está dentro del directorio base
if not canonical.startswith(BASE_DIR + os.sep):
    raise PermissionError('Acceso denegado')

return open(canonical, 'rb').read()
```

8.2 Validación por allowlisting del valor del parámetro (prioridad crítica)

Definir exactamente qué valores son aceptables para el parámetro filename. Rechazar cualquier valor que no cumpla el patrón permitido, sin importar cuánto se ofusque:

```
import re

# Solo permite nombres de archivo con extensiones de imagen conocidas,
# sin barras, puntos iniciales ni caracteres especiales.
ALLOWED = re.compile(r'^[a-zA-Z0-9_-]+\.(jpg|jpeg|png|gif|webp)$')

def validate(filename: str) -> str:
    if not ALLOWED.match(filename):
        raise ValueError('Nombre de archivo no permitido')
    return filename
```

8.3 Por qué el stripping no recursivo es insuficiente

El filtro de este laboratorio falla porque elimina ../ en una sola pasada. Para que el stripping fuera efectivo debería aplicarse de forma recursiva hasta que el resultado no contenga ninguna secuencia de traversal. Sin embargo, incluso el stripping recursivo es insuficiente si no se verifica la ruta canónica resultante, ya que existen técnicas de bypass basadas en codificación URL, rutas absolutas y otros vectores que no contienen ../ explícito:

Técnica de bypass	Ejemplo	¿Lo detiene el stripping recursivo?	¿Lo detiene la canonicalización?
Stripping no recursivo	../../../etc/passwd	No	Sí
Ruta absoluta	/etc/passwd	Sí (no hay ../)	Sí
Codificación URL	..%2f..%2f..%2fetc%2fpasswd	Depende	Sí
Doble codificación	..%252f..%252f..%252fetc%252fpasswd	No	Sí

8.4 Principio de mínimo privilegio

- Ejecutar el servidor web con un usuario sin privilegios que solo tenga permisos de lectura sobre el directorio de archivos estáticos.

- Aplicar controles a nivel del sistema operativo (chroot, namespaces, contenedores) para confinar el proceso del servidor web.

8.5 Diseño seguro del endpoint

- Evitar exponer rutas de archivo en parámetros de URL. Mapear internamente un identificador abstracto (ID numérico o UUID) al archivo real en el servidor, sin que el cliente conozca la ruta.
- Usar funciones del framework para servir archivos estáticos (send_from_directory en Flask, StaticFiles en FastAPI, X-Accel-Redirect en Nginx) que incluyen validación de ruta nativa.

9. Conclusión

La realización de este laboratorio permitió comprender la técnica de bypass mediante stripping no recursivo en un ataque de Path Traversal. La aplicación implementó un filtro que detecta y elimina las secuencias `../` del parámetro filename, pero al hacerlo en una única pasada sin volver a analizar el resultado, dejó abierta la posibilidad de construir un payload anidado (`....//....//....//etc/passwd`) que reconstituye las secuencias de traversal exactamente donde el filtro deja de actuar.

El caso ilustra un principio de seguridad fundamental: los filtros de seguridad basados en la búsqueda y eliminación de patrones conocidos (stripping, blacklisting) son inherentemente frágiles porque su efectividad depende de que el desarrollador anticipe todas las formas posibles de expresar el patrón malicioso. La técnica documentada en este laboratorio es solo una de varias variantes de bypass; existen otras basadas en codificación URL, rutas absolutas, uso de barras invertidas y doble codificación.

El impacto del exploit es el mismo que en casos más simples: lectura no autorizada del archivo `/etc/passwd` con la lista completa de usuarios del sistema operativo del servidor. Esta información puede aprovecharse para escalar el ataque mediante enumeración de cuentas, ataques de fuerza bruta y, en escenarios más complejos, lectura de archivos de configuración con credenciales de acceso.

La única mitigación verdaderamente robusta frente a esta familia de vulnerabilidades es la canonicalización de la ruta resultante combinada con la verificación de confinamiento al directorio base, ya que este enfoque valida el resultado final de la resolución de la ruta sin importar qué técnica de ofuscación empleó el atacante para construir el valor del parámetro.